

tf.compat.v1.distributions.DirichletMultinomial

Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/distributions/DirichletMultinomial

Dirichlet-Multinomial compound distribution.

Function Signatures

```
tf.compat.v1.distributions.DirichletMultinomial( total_count,
concentration, validate_args=False, allow_nan_stats=True,
name='DirichletMultinomial' )
```

Code Examples

```
tf.compat.v1.distributions.DirichletMultinomial(
total_count,
concentration,
validate_args=False,
allow_nan_stats=True,
name='DirichletMultinomial'
)
```

```
tf.compat.v1.distributions.DirichletMultinomial(
total_count,
concentration,
validate_args=False,
allow_nan_stats=True,
name='DirichletMultinomial'
)
```

$$\text{pmf}(n; \alpha, N) = \text{Beta}(\alpha + n) / (\text{prod}_j n_j!) / Z$$

$$Z = \text{Beta}(\alpha) / N!$$

$$\text{pmf}(n; \alpha, N) = \text{Beta}(\alpha + n) / (\text{prod}_j n_j!) / Z$$

$$Z = \text{Beta}(\alpha) / N!$$

```
K <= min(2**31-1, {
tf.float16: 2**11,
tf.float32: 2**24,
tf.float64: 2**53 }[param.dtype])
```

```
K <= min(2**31-1, {
tf.float16: 2**11,
tf.float32: 2**24,
tf.float64: 2**53 }[param.dtype])
```

```
alpha = [1., 2., 3.]
n = 2.
dist = DirichletMultinomial(n, alpha)
```

```
alpha = [1., 2., 3.]
n = 2.
dist = DirichletMultinomial(n, alpha)
```

Documentation

Dirichlet-Multinomial compound distribution.

Inherits From: Distribution

The Dirichlet-Multinomial distribution is parameterized by a (batch of) length- K concentration vectors ($K > 1$) and a `total_count` number of trials, i.e., the number of trials per draw from the DirichletMultinomial. It is defined over a (batch of) length- K vector counts such that `tf.reduce_sum(counts, -1) = total_count`. The Dirichlet-Multinomial is identically the Beta-Binomial distribution when $K = 2$.

Mathematical Details

The Dirichlet-Multinomial is a distribution over K -class counts, i.e., a length- K vector of non-negative integer counts $\mathbf{n} = [n_0, \dots, n_{K-1}]$.

The probability mass function (pmf) is,

where:

- $\text{concentration} = \alpha = [\alpha_0, \dots, \alpha_{K-1}]$, $\alpha_j > 0$,
- $\text{total_count} = N$, N a positive integer,
- $N!$ is N factorial, and,
- $\text{Beta}(\mathbf{x}) = \prod_j \text{Gamma}(x_j) / \text{Gamma}(\sum_j x_j)$ is the

multivariate beta function,

and,

- Gamma is the gamma function.

Dirichlet-Multinomial is a compound distribution, i.e., its samples are generated as follows.

- Choose class probabilities:

$\text{probs} = [p_0, \dots, p_{K-1}] \sim \text{Dir}(\text{concentration})$

- Draw integers:

$\text{counts} = [n_0, \dots, n_{K-1}] \sim \text{Multinomial}(\text{total_count}, \text{probs})$

The last concentration dimension parametrizes a single Dirichlet-Multinomial distribution. When calling distribution functions (e.g., `dist.prob(counts)`), `concentration`, `total_count` and `counts` are broadcast to the same shape. The last dimension of `counts` corresponds single Dirichlet-Multinomial distributions.

Distribution parameters are automatically broadcast in all functions; see examples for details.

Pitfalls

The number of classes, `K`, must not exceed:

- the largest integer representable by `self.dtype`, i.e.,

$2^{(\text{mantissa_bits}+1)}$ (IEEE754),

- the maximum Tensor index, i.e., $2^{31}-1$.

In other words,

Examples

Creates a 3-class distribution, with the 3rd class is most likely to be drawn.

The distribution functions can be evaluated on counts.

Creates a 2-batch of 3-class distributions.

Attributes

Stats return +/- infinity when it makes sense. E.g., the variance of a Cauchy distribution is infinity. However, sometimes the statistic is undefined, e.g., if a distribution's pdf does not achieve a maximum within the support of the distribution, the mode is undefined. If the mean is undefined, then by definition the variance is undefined. E.g. the mean for Student's T for $df = 1$ is undefined (no clear way to say it is either + or - infinity), so the variance = $E[(X - \text{mean})^2]$ is also undefined.

May be partially defined or unknown.

The batch dimensions are indexes into independent, non-identical parameterizations of this distribution.

May be partially defined or unknown.

Currently this is one of the static instances

distributions.FULLY_REPARAMETERIZED
or distributions.NOT_REPARAMETERIZED.

Methods

batch_shape_tensor

[View source](#)

Shape of a single sample from a single event index as a 1-D Tensor.

The batch dimensions are indexes into independent, non-identical parameterizations of this distribution.

[View source](#)

Cumulative distribution function.

Given random variable X , the cumulative distribution function cdf is:

[View source](#)

Creates a deep copy of the distribution.

covariance

[View source](#)

Covariance.

Covariance is (possibly) defined only for non-scalar-event distributions.

For example, for a length- k , vector-valued distribution, it is calculated as,

where Cov is a (batch of) $k \times k$ matrix, $0 \leq (i, j) < k$, and E denotes expectation.

Alternatively, for non-vector, multivariate distributions (e.g., matrix-valued, Wishart), Covariance shall return a (batch of) matrices under some vectorization of the events, i.e.,

where Cov is a (batch of) $k' \times k'$ matrices, $0 \leq (i, j) < k' = \text{reduce_prod}(\text{event_shape})$, and Vec is some function mapping indices of this distribution's event dimensions to indices of a length- k' vector.

Additional documentation from DirichletMultinomial:

The covariance for each batch member is defined as the following:

where concentration = alpha and
total_concentration = alpha_0 = sum_j alpha_j.

The covariance between elements in a batch is defined as:

cross_entropy

[View source](#)

Computes the (Shannon) cross entropy.

Denote this distribution (self) by P and the other distribution by Q . Assuming P, Q are absolutely continuous with respect to one another and permit densities $p(x) dx$ and $q(x) dx$, (Shanon)

cross entropy is defined as:

where F denotes the support of the random variable $X \sim P$.

entropy

[View source](#)

Shannon entropy in nats.

event_shape_tensor

[View source](#)

Shape of a single sample from a single batch as a 1-D int32 Tensor.

is_scalar_batch

[View source](#)

Indicates that `batch_shape == []`.

is_scalar_event

[View source](#)

Indicates that `event_shape == []`.

kl_divergence

[View source](#)

Computes the Kullback--Leibler divergence.

Denote this distribution (self) by p and the other distribution by

q. Assuming p, q are absolutely continuous with respect to reference measure r , the KL divergence is defined as:

where F denotes the support of the random variable $X \sim p$, $H[., .]$ denotes (Shanon) cross entropy, and $H[.]$ denotes (Shanon) entropy.

log_cdf

[View source](#)

Log cumulative distribution function.

Given random variable X , the cumulative distribution function cdf is:

Often, a numerical approximation can be used for $\log_cdf(x)$ that yields a more accurate answer than simply taking the logarithm of the cdf when $x \ll -1$.

log_prob

[View source](#)

Log probability density/mass function.

Additional documentation from `DirichletMultinomial`:

For each batch of counts,
value = [n_0, ..., n_{K-1}], $P[\text{value}]$ is the probability that after sampling `self.total_count` draws from this Dirichlet-Multinomial distribution, the number of draws falling in class j is n_j . Since this definition is exchangeable;

different sequences have the same counts so the probability includes a combinatorial coefficient.

log_survival_function

[View sour](#)